

```

rm(list = ls())

# Local github
codepath <- c("/media/larryleon/My Projects/GitHub/Forest-Search/R/")
source(paste0(codepath, "source_forestsearch_v0.R"))
source_fs_functions(file_loc = codepath)

library(kableExtra)
library(knitr)
library(ggplot2)
library(gridExtra)
library(cubature)
library(aVirtualTwins)
library(randomForest)
library(survival)
library(survminer)
library(grf)
library(policytree)
library(data.table)
library(plyr)
library(dplyr)
library(glmnet)
library(corrplot)

library(table1)

maxFollow <- 84
cens.type <- "weibull"

# m1 -censoring adjustment
muC.adj <- log(1.5)
k.z3 <- 1
k.treat <- 0.9

z1_frac <- 0.25 # Default model index 'm1' (The 1st quartile of z1=er)
pH_super <- 0.125 # non-NULL re-defines z1_frac

if (is.null(pH_super)) {
  # pH_check<-with(gbsg,mean(pgr<=quantile(pgr,c(z3_frac),1,0) &
  # er<=quantile(er,z1_frac)))
  pH_check <- with(gbsg, mean(meno == 0 & er <= quantile(er, z1_frac)))
  cat("Underlying pH_super", c(pH_check), "\n")
}
# pH_super specified If pH_super then override z1_frac and find z1_frac to
# yield pH_super

if (!is.null(pH_super)) {
  # Approximate Z1 quantile to yield pH proportion
  z1_q <- uniroot(propH.obj4, c(0, 1), tol = 1e-04, pH.target = pH_super)$root
  # pH_check<-with(gbsg,mean(pgr<=quantile(pgr,c(z3_frac),1,0) &
  # er<=quantile(er,z1_q)))
  pH_check <- with(gbsg, mean(meno == 0 & er <= quantile(er, z1_q)))
  cat("pH", c(pH_check), "\n")
  rel_error <- (pH_super - pH_check)/pH_super
  if (abs(rel_error) >= 0.1)

```

```

    stop("pH_super approximation relative error exceeds 10%")
    z1_frac <- z1_q
    cat("Underlying pH_super", c(pH_check), "\n")
  }

## pH 0.122449
## Underlying pH_super 0.122449

# Bootstrap on log(hr) scale converted to HR (est.loghr=TRUE & est.scale='hr')
t.start.all <- proc.time()[3]

##### Forest search criteria
hr.threshold <- 1.25 # Initial candidates
hr.consistency <- 1 # Candidates for many splits
pconsistency.threshold <- 0.9
stop.threshold <- 0.95
maxk <- 2
nmin.fs <- 60
pstop_futile <- 0.5
# Limit timing for forestsearch
max.minutes <- 3
m1.threshold <- Inf # Turning this off (Default)
# pconsistency.threshold<-0.70 # Minimum threshold (will choose max among
# subgroups satisfying)
fs.splits <- 400 # How many times to split for consistency
# vi is % factor is selected in cross-validation --> higher more important
vi.grf.min <- 0.2
# Null, turns off grf screening
d.min <- 10 # Min number of events for both arms (d0.min=d1.min=d.min)
# default=5 Virtual twins analysis Counter-factual difference (C-E) >=
# vt.threshold Large values in favor of C (control)
vt.threshold <- 0.225 # For VT delta
treat.threshold <- 0

maxdepth <- 2
n.min <- 60
ntree <- 1000
# GRF criteria
dmin.grf <- 12 # For GRF delta
# Note: For CRT this represents dmin.grf/2 RMS for control (-dmin.grf/2 for
# treatment)
frac.tau <- 0.6

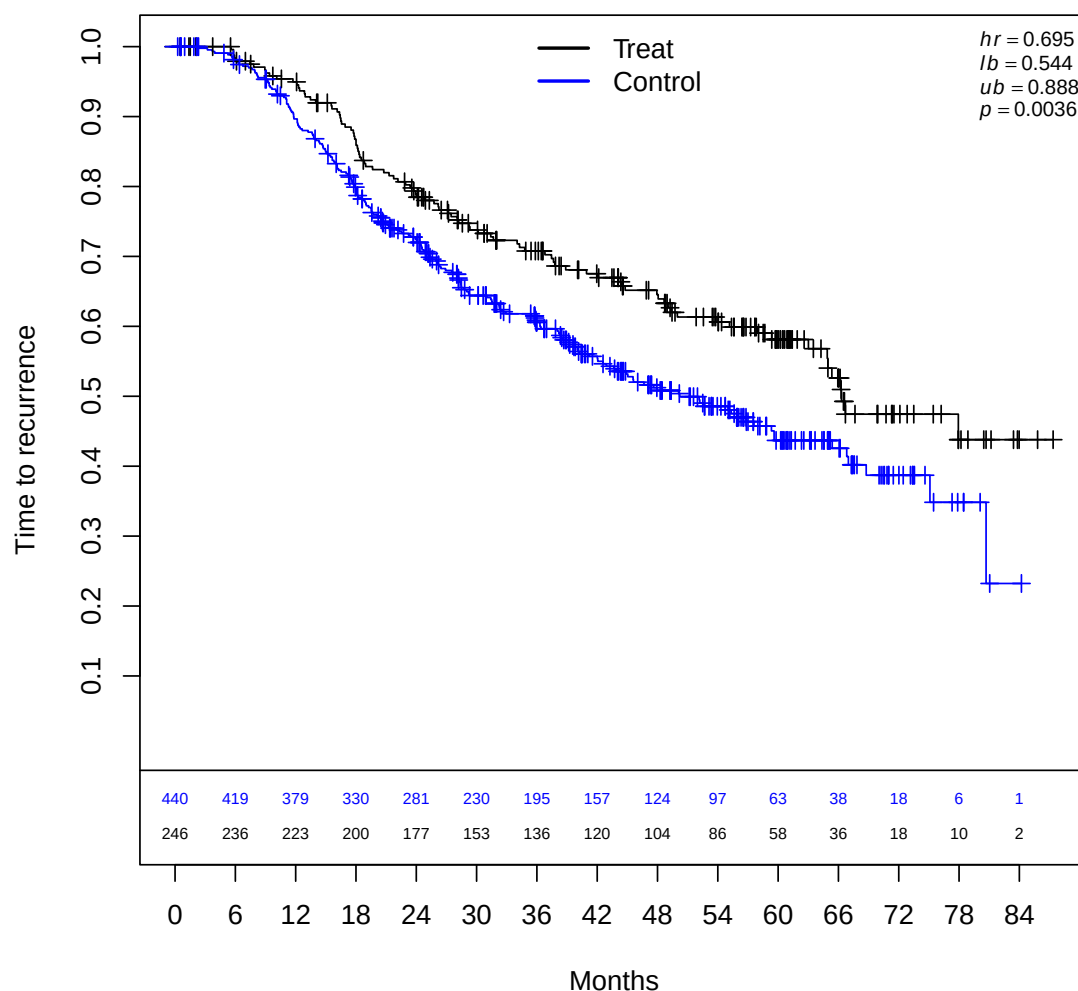
outcome.name <- c("y.sim")
event.name <- c("event.sim")
id.name <- c("id")
treat.name <- c("treat")

cox.formula.sim <- as.formula(paste("Surv(y.sim,event.sim)~treat"))
cox.formula.adj.sim <- as.formula(paste("Surv(y.sim,event.sim)~treat+v1+v2+v3+v4+v5"))

mod.harm <- "alt"
hrH.target <- 2
# out.loc = NULL turns off file creation
N <- 1000

```

```
this.dgm <- get.dgm4(mod.harm = mod.harm, N = N, k.treat = k.treat, hrH.target = hrH.target,
  cens.type = cens.type, out.loc = NULL, details = TRUE, parms_torand = FALSE)
```



```
## Super-population empirical harm and non-harm hazard ratios= 2.000007 0.6466405
## Causal HR (empirical ITT)= 0.7057463
```

```
# sim sensH ppvH sensHc ppvHc 1: 11 0.5000000 0.5731707 0.9422442 0.9239482 2:
# 14 0.6891892 0.5666667 0.9376997 0.9622951 3: 17 0.3296703 0.2830189
# 0.8752053 0.8973064 4: 18 0.4489796 0.4943820 0.9252492 0.9116203 5: 20
# 0.8045977 0.4347826 0.8515498 0.9684601 6: 23 0.8101266 0.5203252 0.9049919
# 0.9740035 7: 30 0.7802198 0.5182482 0.8916256 0.9644760
```

```
file_out <- c("output/sim-FS4_sim20.Rdata")
```

```
n_add_noise <- 3
```

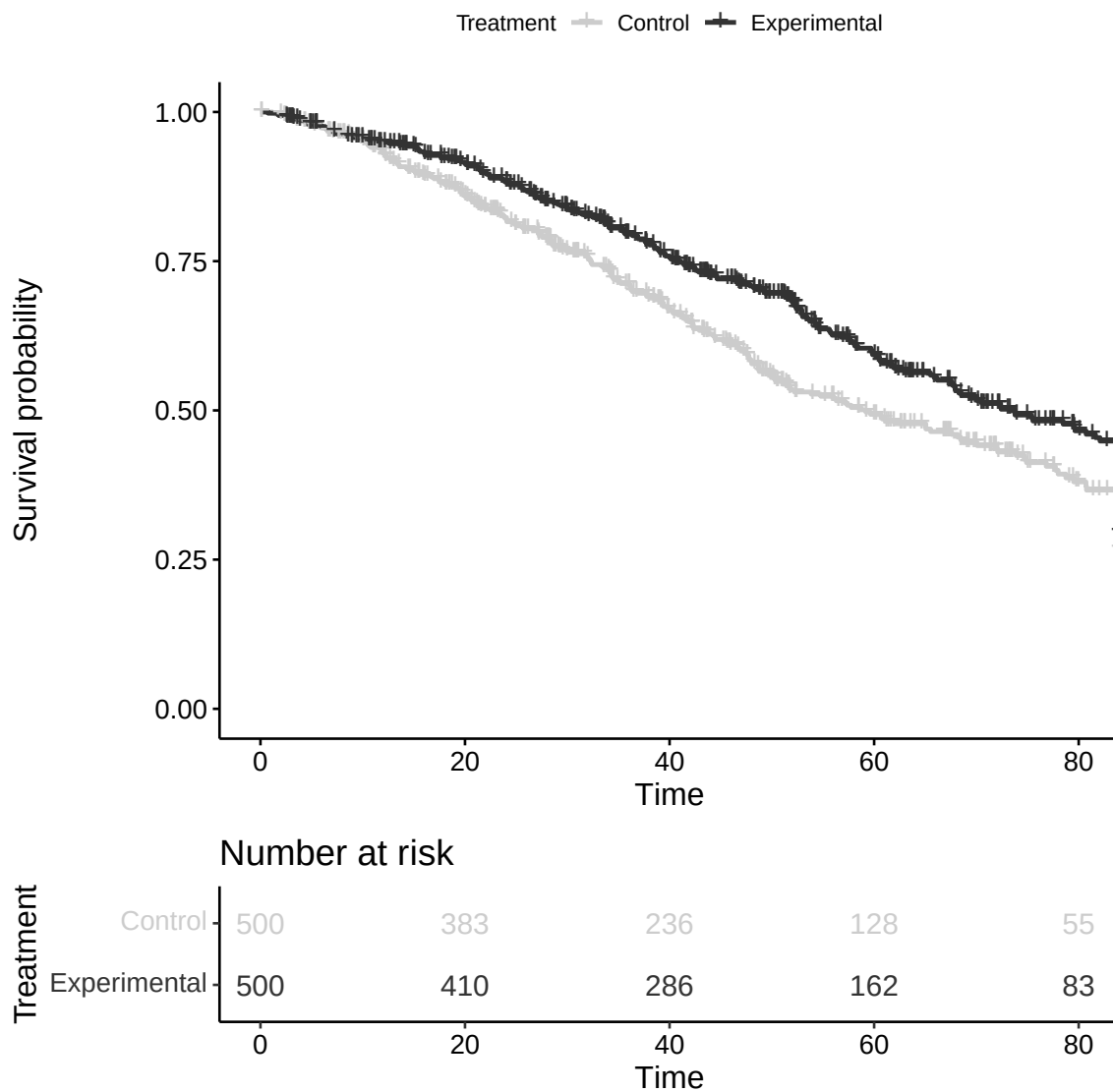
```
sim <- 20
```

```
dgm <- this.dgm$dgm

x <- sim_aftm4_gbsg(dgm = dgm, n = N, maxFollow = maxFollow, muC.adj = muC.adj, simid = sim)

kmfit <- survfit(Surv(y.sim, event) ~ treat, data = x)

ggsurvplot(kmfit, data = x, main = "K-M curves for simulated data", legend = "top",
  legend.title = "Treatment", legend.labs = c("Control", "Experimental"), palette = "grey",
  risk.table = TRUE, risk.table.col = "strata")
```



```
if (n_add_noise == 0) {
  confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade3")
}

if (n_add_noise == 5) {
  set.seed(8316951 + 1000 * sim)
  # Add 5 noise
  x$noise1 <- rnorm(N)
```

```

x$noise2 <- rnorm(N)
x$noise3 <- rnorm(N)
x$noise4 <- rnorm(N)
x$noise5 <- rnorm(N)
confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade3", "noise1",
  "noise2", "noise3", "noise4", "noise5")
}

if (n_add_noise == 3) {
  set.seed(8316951 + 1000 * sim)
  # Add 3 noise
  x$noise1 <- rnorm(N, sd = 1)
  x$noise2 <- rnorm(N, sd = 1)
  x$noise3 <- rnorm(N, sd = 1)
  confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade3", "noise1",
    "noise2", "noise3")
}

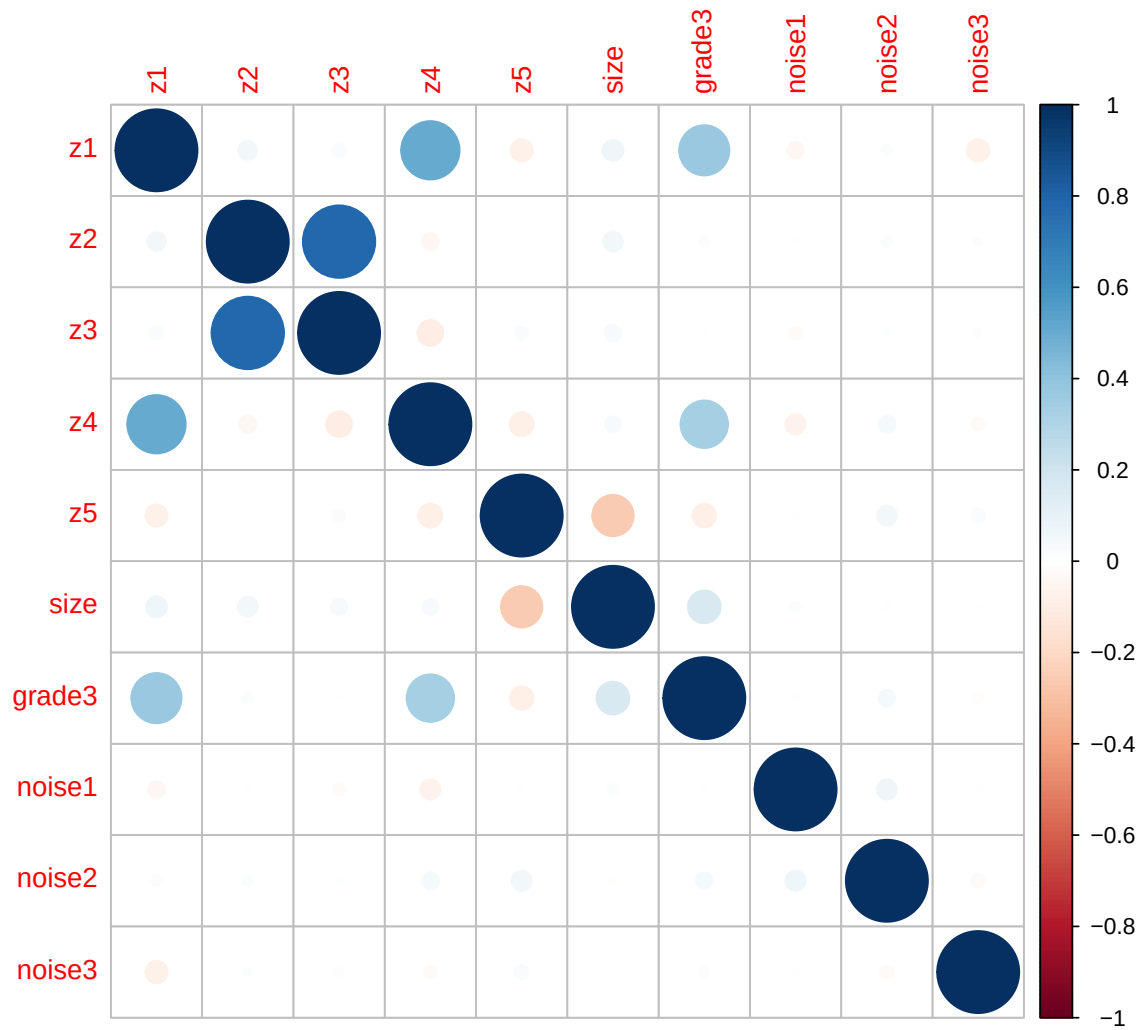
# More challenging which_replace <- which(confounders.name=='z1')
# confounders.name[which_replace] <- 'er'

cox.formula.check <- as.formula(paste("Surv(y.sim,event.sim)~treat+z1+z2+z3+z4+z5+size+grade3+noise1
coxph(cox.formula.check, data = x)

## Call:
## coxph(formula = cox.formula.check, data = x)
##
##              coef exp(coef) se(coef)      z      p
## treat  -0.511038  0.599873  0.086640  -5.898 3.67e-09
## z1       0.356189  1.427877  0.106742   3.337 0.000847
## z2      -0.529682  0.588792  0.141726  -3.737 0.000186
## z3       0.260957  1.298172  0.145595   1.792 0.073076
## z4       0.506190  1.658958  0.102723   4.928 8.32e-07
## z5      -0.980474  0.375133  0.090605 -10.821 < 2e-16
## size     0.003142  1.003147  0.002804   1.121 0.262478
## grade3  -0.073187  0.929427  0.102507  -0.714 0.475242
## noise1  -0.013427  0.986662  0.044859  -0.299 0.764693
## noise2   0.082755  1.086276  0.043782   1.890 0.058733
## noise3   0.021561  1.021795  0.041955   0.514 0.607326
##
## Likelihood ratio test=246.2 on 11 df, p=< 2.2e-16
## n= 1000, number of events= 557

Zm <- cor(as.matrix(x[, c(confounders.name)]))
corrplot(Zm)

```



```
suppressWarnings(table1(~z1 + z2 + z3 + z4 + z5 + size + grade3 + grade + noise1 +
  noise2 + noise3 | treat, data = x))
```

	0	1	Overall
	(N=500)	(N=500)	(N=1000)
z1			
Mean (SD)	0.276 (0.447)	0.300 (0.459)	0.288 (0.453)
Median [Min, Max]	0 [0, 1.00]	0 [0, 1.00]	0 [0, 1.00]
z2			
Mean (SD)	0.524 (0.500)	0.518 (0.500)	0.521 (0.500)
Median [Min, Max]	1.00 [0, 1.00]	1.00 [0, 1.00]	1.00 [0, 1.00]
z3			
Mean (SD)	0.424 (0.495)	0.422 (0.494)	0.423 (0.494)
Median [Min, Max]	0 [0, 1.00]	0 [0, 1.00]	0 [0, 1.00]
z4			
Mean (SD)	0.506 (0.500)	0.522 (0.500)	0.514 (0.500)
Median [Min, Max]	1.00 [0, 1.00]	1.00 [0, 1.00]	1.00 [0, 1.00]
z5			
Mean (SD)	0.532 (0.499)	0.528 (0.500)	0.530 (0.499)
Median [Min, Max]	1.00 [0, 1.00]	1.00 [0, 1.00]	1.00 [0, 1.00]
size			
Mean (SD)	28.8 (14.5)	29.8 (15.0)	29.3 (14.8)
Median [Min, Max]	25.0 [3.00, 100]	25.0 [3.00, 100]	25.0 [3.00, 100]
grade3			
Mean (SD)	0.288 (0.453)	0.250 (0.433)	0.269 (0.444)
Median [Min, Max]	0 [0, 1.00]	0 [0, 1.00]	0 [0, 1.00]
grade			
Mean (SD)	2.18 (0.604)	2.14 (0.588)	2.16 (0.596)
Median [Min, Max]	2.00 [1.00, 3.00]	2.00 [1.00, 3.00]	2.00 [1.00, 3.00]
noise1			
Mean (SD)	-0.0633 (0.972)	0.0133 (0.962)	-0.0250 (0.967)
Median [Min, Max]	-0.0662 [-3.20, 2.92]	-0.0467 [-2.86, 3.29]	-0.0595 [-3.20, 3.29]
noise2			
Mean (SD)	-0.0485 (0.983)	-0.000473 (0.937)	-0.0245 (0.960)
Median [Min, Max]	-0.00414 [-3.25, 2.80]	0.0279 [-2.37, 2.52]	0.0170 [-3.25, 2.80]
noise3			
Mean (SD)	0.0597 (1.07)	0.0724 (1.00)	0.0661 (1.04)
Median [Min, Max]	0.108 [-3.38, 3.09]	0.152 [-2.55, 3.65]	0.129 [-3.38, 3.65]

Options Allconfounders.name is list of confounders within analysis dataset
(1) use_lasso=TRUE & use_grf=FALSE Lasso used to possibly reduce dimension
Any continuous factors are cut at medians (2) use_lasso=TRUE & use_grf=TRUE
Lasso used to reduce dimension Continuous covariates are cut at medians
However, if GRF selects a covariate cut then only that cut is used: For
example if 'age <= median(ag)' is called for per Lasso but GRF includes 'age
<= 54', then only the latter is used (3) use_grf_only = TRUE (overrides
use_lasso and use_grf) Only factors selected via GRF are used (4) use_lasso =
F & use_grf = T Median cuts (unless selected via GRF) as in (2) However no
possible dimension reduction via lasso All categorical factors included

```
use_lasso <- TRUE
use_grf <- TRUE
use_grf_only <- FALSE
```

```

fs.est <- forestsearch(df.analysis = x, Allconfounders.name = confounders.name, details = TRUE,
  use_lasso = use_lasso, use_grf = use_grf, use_grf_only = use_grf_only, dmin.grf = 12,
  frac.tau = 0.6, conf_force = NULL, outcome.name = outcome.name, treat.name = treat.name,
  event.name = event.name, id.name = id.name, n.min = nmin.fs, hr.threshold = hr.threshold,
  hr.consistency = hr.consistency, fs.splits = fs.splits, d0.min = d.min, d1.min = d.min,
  pstop_futile = pstop_futile, pconsistency.threshold = pconsistency.threshold,
  stop.threshold = stop.threshold, max.minutes = max.minutes, maxk = maxk, by.risk = 12,
  plot.sg = TRUE, vi.grf.min = vi.grf.min)

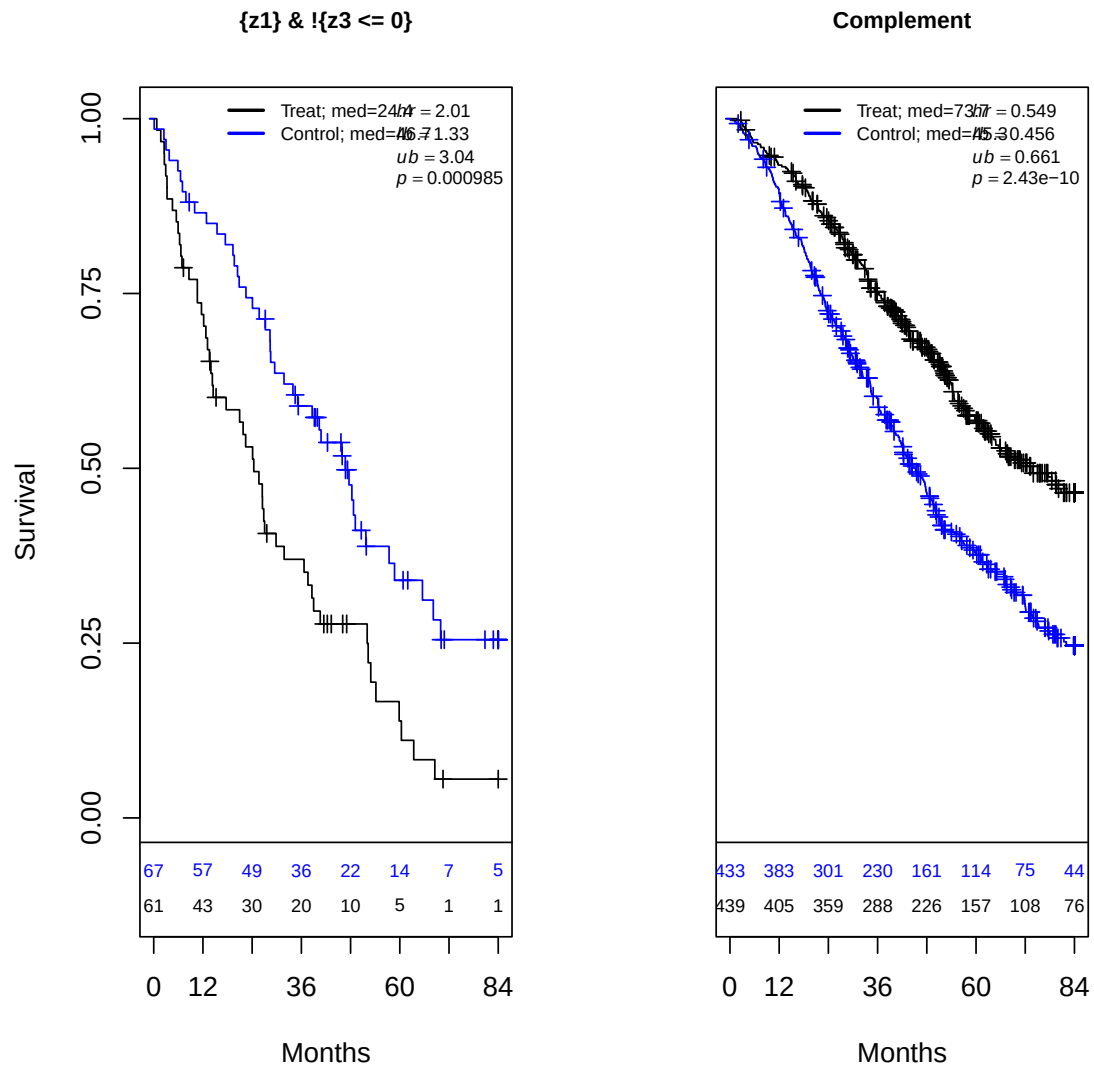
## tau, maxdepth= 48.43595 2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 1         2  -3.6776039  991.0000000  0.8985883   3.6776039  991.0000000
## 11        4  -5.6613665  654.0000000  1.0446868   5.6613665  654.0000000
## 3         6  -7.2408361  160.0000000  2.2506763   7.2408361  160.0000000
## 4         7   9.1461201  128.0000000  2.9304296  -9.1461201  128.0000000
##   treated.se      diff Nsg depth
## 1  0.8985883  -7.355208  991     1
## 11 1.0446868 -11.322733  654     2
## 3  2.2506763 -14.481672  160     2
## 4  2.9304296  18.292240  128     2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 4         7   9.14612   128.00000   2.93043   -9.14612   128.00000
##   treated.se      diff Nsg depth
## 4  2.93043 18.29224 128     2
## [1] "z1 <= 0"          "noise2 <= 1.28" "z3 <= 0"
## [1] "z3 <= 0"
## # of continuous/categorical characteristics 4 6
## Continuous characteristics: size noise1 noise2 noise3
## Categorical characteristics: z1 z2 z3 z4 z5 grade3
## 10 x 1 sparse Matrix of class "dgCMatrix"
##               s0
## z1      0.2949221166
## z2     -0.2137814072
## z3      .
## z4      0.4313498790
## z5     -0.8768762261
## size    0.0005584749
## grade3   .
## noise1   .
## noise2  0.0381326853
## noise3   .
## Cox-LASSO selected: z1 z2 z4 z5 size noise2
## Cox-LASSO not selected: z3 grade3 noise1 noise3
## Median cuts (after Lasso if use_lasso) size noise2
## Categorical (after Lasso if use_lasso) z1 z2 z4 z5
## Prior to removing (if in GRF): size noise2
## Left (to cut at median) after removing others if in GRF: size
## Initial GRF cuts included z1 <= 0 noise2 <= 1.28 z3 <= 0
## # of candidate subgroup factors= 7
## [1] "size <= median(size)" "noise2 <= 1.28"      "z1"
## [4] "z2"                  "z4"          "z5"
## [7] "z3 <= 0"
## Confounders per grf screening q3 q7 q6 q1 q4 q2 q5
##   FSconfounders.name      vi.cs vi_ratio
## 3                   q3 0.24355784 1.7049049
## 7                   q7 0.23642820 1.6549974

```

```

## 6          q6 0.17530979 1.2271685
## 1          q1 0.09977403 0.6984182
## 4          q4 0.08393604 0.5875523
## 2          q2 0.08231087 0.5761761
## 5          q5 0.07868323 0.5507826
## Number of unique levels (L) and possible subgroups= 14 16383
## # of subgroups based on # variables > k.max and excluded (per million) 0.016278
## k.max= 2
## Events criteria for control,exp= 10 10
## # of subgroups with events less than criteria: control, experimental 14 14
## # of subgroups with sample size less than criteria 21
## # of subgroups meeting all criteria = 84
## # of subgroups fitted (Cox model estimable) = 84
## *Subgroup Searching Minutes=* 0.004883333
## Number of subgroups meeting HR threshold 1
## # of candidate subgroups (meeting HR criteria) = 1
## Subgroups (1st 10) meeting overall screening thresholds (HR, m1) sorted by HRs= Inf
##      K   n   E d1   m1   m0   HR L(HR) U(HR) q3.0 q3.1 q7.0 q7.1 q6.0 q6.1 q1.0
## 1:  2 128 92 50 24.4 46.72 2.01  1.33  3.04   0   1   1   0   0   0   0
## 2:  NA  NA  NA  NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
## 3:  NA  NA  NA  NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
## 4:  NA  NA  NA  NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
## 5:  NA  NA  NA  NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
## 6:  NA  NA  NA  NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
## 7:  NA  NA  NA  NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
## 8:  NA  NA  NA  NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
## 9:  NA  NA  NA  NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
## 10: NA  NA  NA  NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
##      q1.1 q4.0 q4.1 q2.0 q2.1 q5.0 q5.1
## 1:    0    0    0    0    0    0    0
## 2:   NA   NA   NA   NA   NA   NA   NA
## 3:   NA   NA   NA   NA   NA   NA   NA
## 4:   NA   NA   NA   NA   NA   NA   NA
## 5:   NA   NA   NA   NA   NA   NA   NA
## 6:   NA   NA   NA   NA   NA   NA   NA
## 7:   NA   NA   NA   NA   NA   NA   NA
## 8:   NA   NA   NA   NA   NA   NA   NA
## 9:   NA   NA   NA   NA   NA   NA   NA
## 10:  NA   NA   NA   NA   NA   NA   NA
## Consistency 1
## # of splits= 400
## Model, % Consistency Met= {z1} !{z3 <= 0} 1
## Number of subgroups meeting consistency criteria=
##      p.consistency Nsg group.id m.index K   M.1          M.2
## 1:          1 128          1          1 2 {z1} !{z3 <= 0}

```



```
## [1] "{z1}"      "!("{z3 <= 0}"
## % consistency criteria met= 1
## SG focus= hr
## Subgroup Consistency Minutes= 0.02353333
## Subgroup found
## Minutes overall= 0.05728333

grf.est <- grf.subg.harm.survival(data = x, confounders.name = confounders.name,
  outcome.name = outcome.name, event.name = event.name, id.name = id.name, treat.name = treat.name,
  n.min = n.min, dmin.grf = dmin.grf, frac.tau = 1, details = TRUE)

## tau, maxdepth= 80.72659 2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 1         2   -9.007304   961.000000   1.655284    9.007304   961.000000
## 11        4  -12.249489   654.000000   1.966338   12.249489   654.000000
## 3         6  -16.700336   160.000000   3.854013   16.700336   160.000000
## 4         7   15.193675   128.000000   4.590477  -15.193675   128.000000
##   treated.se      diff Nsg depth
## 1    1.655284 -18.01461 961     1
```

```
## 11 1.966338 -24.49898 654 2
## 3 3.854013 -33.40067 160 2
## 4 4.590477 30.38735 128 2
## leaf.node control.mean control.size control.se treated.mean treated.size
## 4 7 15.193675 128.000000 4.590477 -15.193675 128.000000
## treated.se diff Nsg depth
## 4 4.590477 30.38735 128 2
## [1] "z1 <= 0" "noise2 <= 1.28" "z3 <= 0"
## [1] "z3 <= 0"
```

```
library(doParallel)
registerDoParallel(parallel::detectCores(logical = FALSE))

cox.formula.boot <- cox.formula.sim
max.minutes <- 7
# Suggest running 50, first ... to get timing estimate

NB <- 1000

df_boot_analysis <- fs.est$df.est

fitH <- get_Cox_sg(df_sg = subset(df_boot_analysis, treat.recommend == 0), cox.formula = cox.formula)
H_obs <- fitH$est_obs # log(hr) scale
seH_obs <- fitH$se_obs
# Hc observed estimates
fitHc <- get_Cox_sg(df_sg = subset(df_boot_analysis, treat.recommend == 1), cox.formula = cox.formula)
Hc_obs <- fitHc$est_obs
seHc_obs <- fitHc$se_obs
rm("fitH", "fitHc")

Ystar_mat <- bootYstar({
  ystar <- get_Ystar(boot)
}, boots = NB, seed = 8316951, counter = "boot", export = fun_arg_list_boot)
# Check dimension
if (dim(Ystar_mat)[1] != NB | dim(Ystar_mat)[2] != nrow(df_boot_analysis)) stop("Dimension of Ystar_mat")

tB.start <- proc.time()[3]
# Bootstraps
resB <- bootPar({
  ans <- fsboot_forparallel(boot)
}, boots = NB, seed = 8316951, counter = "boot", export = fun_arg_list_boot)
tB.now <- proc.time()[3]
tB.min <- (tB.now - tB.start)/60

doParallel::stopImplicitCluster()

cat("Minutes for Boots", c(NB, tB.min), "\n")
## Minutes for Boots 1000 4.489017

cat("Projection per 1000", c(tB.min * (1000/NB)), "\n")
## Projection per 1000 4.489017

cat("Propn bootstrap subgroups found =", c(sum(!is.na(resB$H_biasadj_1))/NB), "\n")
## Propn bootstrap subgroups found = 0.926
```

```

# How many timed out
cat("Number timed out=", c(sum(is.na(resB$H_biasadj_1) & resB$tmins_search > max.minutes)),
    "\n")

## Number timed out= 0

H_estimates <- get_dfRes(Hobs = H_obs, seHobs = seH_obs, H1_adj = resB$H_biasadj_2,
    ystar = Ystar_mat, cov_method = "standard", cov_trim = 0)
Hc_estimates <- get_dfRes(Hobs = Hc_obs, seHobs = seHc_obs, H1_adj = resB$Hc_biasadj_2,
    ystar = Ystar_mat, cov_method = "standard", cov_trim = 0)

print(H_estimates)
##           H0          sdH0 H0_lower H0_upper          H1          sdH1 H1_lower H1_upper
## 1: 2.008805 0.4252793 1.326575 3.04189 1.742604 0.4009083 1.110118 2.735448
print(Hc_estimates)
##           H0          sdH0 H0_lower H0_upper          H1          sdH1 H1_lower
## 1: 0.5491568 0.05198552 0.4561608 0.6611115 0.5507616 0.07380894 0.4235376
##           H1_upper
## 1: 0.7162016

# H_estimates<-get_dfRes(Hobs=H_obs,seHobs=seH_obs,H1_adj=resB$H_biasadj_1,ystar=Ystar_mat,cov_metho
# Hc_estimates<-get_dfRes(Hobs=Hc_obs,seHobs=seHc_obs,H1_adj=resB$Hc_biasadj_1,ystar=Ystar_mat,cov_m
# print(H_estimates) print(Hc_estimates)

bootit <- list(H_estimates = H_estimates, Hc_estimates = Hc_estimates)
ans3 <- fsBoot.parallel.out(df = df_boot_analysis, FSboots = bootit, cox.formula.boot = cox.formula.
    sim_number = sim)
tall.min <- (tB.now - t.start.all)/60
cat("Overall minutes for analysis", c(tall.min), "\n")
## Overall minutes for analysis 4.625617

save(dgm, x, fs.est, grf.est, bootit, ans3, tall.min, resB, cox.formula.boot, file = file_out)

# ITT causal hazard ratio
mean(fs.est$df.est$hlin.ratio)
## [1] 0.8146543

# H Oracle
dfH <- subset(fs.est$df.est, flag.harm == 1)
mean(dfH$hlin.ratio)
## [1] 2.249393

dfH_est <- subset(fs.est$df.est, treat.recommend == 0)
mean(dfH_est$hlin.ratio)
## [1] 2.249393

# Hc Oracle
dfHc <- subset(fs.est$df.est, flag.harm == 0)
mean(dfHc$hlin.ratio)
## [1] 0.6040505

dfHc_est <- subset(fs.est$df.est, treat.recommend == 1)
mean(dfHc_est$hlin.ratio)
## [1] 0.6040505

# dfSens <- subset(fs.est$df.est, flag.harm==1 & treat.recommend==0) sensH <-
# nrow(dfSens)/nrow(dfH)

```

Table 1 Simulated data example: Cox hazard ratio (hr) estimates for the ITT population and subgroups H and H^c . Cox model estimates are based on subgroups: H true (knowing the actual subgroup, a-priori); the estimated subgroup $\hat{H}$; and the bootstrap (1,000 resamples) bias-correction to $\hat{H}$ estimates, denoted $\hat{H}_{bc}$. Estimates for the complement H^c are defined analogously. The number of subjects in each population (# Subjects) and the number of subjects correctly classified (# Correct) in the true subgroups H and H^c , respectively, are listed.

	HR Estimate	Lower	Upper	# Subjects	# Correct
ITT	0.652	0.551	0.771	1000	.
H subgroup estimates					
H_{oracle}	2.009	1.327	3.041	128	.
$\hat{H}$	2.009	1.327	3.041	128	128
$\hat{H}_{bc}$	1.743	1.11	2.736	128	128
H-complement subgroup estimates					
H_{oracle}^c	0.549	0.456	0.661	872	.
$\hat{H}^c$	0.549	0.456	0.661	872	872
$\hat{H}_{bc}^c$	0.551	0.423	0.717	872	872

```
## Confounders evaluated in Forestsearch:
## [1] "size <= median(size)" "noise2 <= 1.28"      "z1"
## [4] "z2"                  "z4"                  "z5"
## [7] "z3 <= 0"
```